

ReduLink: Authenticated Reference Substitution for Redundancy-Suppressed Transfers over Encrypted QUIC Streams

Michél Nguyen

University of the People | minh.systems | ORCID: 0000-0001-6834-4422

Abstract

Encrypted transports such as QUIC deliberately hide payload bytes from the network, which disables the transparent in-network redundancy elimination that wide-area optimizers have historically relied on. ReduLink is an endpoint-controlled representation layer that recovers redundancy savings without breaking encryption: it replaces repeated payload chunks with compact references that are individually authenticated against epoch, scope, stream identifier, reconstructed offset, length, nonce, and chunk identity. The contribution is not a new chunk-matching algorithm and not a claim of faster physical links; it is authenticated, scoped, QUIC-compatible reference substitution with explicit reconstruction, privacy scoping, and fail-closed semantics.

The artifact maps compact binary ReduLink messages into TLS-protected aioquic QUIC streams, implements semantic MISS/FULL repair, derives keys with an exporter-style schedule, and tests replay and tamper rejection. We evaluate against fixed-block reuse, gzip/zstd, and real rsync over deterministic journal fixtures, hash-pinned public source releases, object-aligned public-release transfers, and a layer-like positive case, and we report native QUIC stream experiments under datagram loss, scaling, repeated trials, block-size sensitivity, component costs, competing-flow and bottleneck-emulation fairness, and conservative wire-byte accounting.

Quantitatively, on object-aligned public release transfers ReduLink reaches 1.94x to 9.70x effective stream-payload reduction (and 21.37x on a layer-like positive case), within a few percent of an idealized fixed-object-reuse baseline, while the authenticated variant stays within a few additional percent; on ordinary source-tree updates rsync wins by up to 73x, which we report rather than hide. Native aioquic experiments confirm byte-exact reconstruction with deterministic stream multipliers under zero and periodic datagram loss, and the accounting experiments confirm that congestion and fairness are charged on encoded bytes, not reconstructed bytes. The result is a clear deployment boundary: simple reuse or rsync wins for file-tree synchronization, while ReduLink is preferable for object-aligned or layer-like warm-state transfers where an authenticated, stream-compatible reference is the desired abstraction.

Keywords: QUIC; HTTP/3; redundancy elimination; data deduplication; content-defined chunking; shared dictionaries; authenticated references; encrypted transport; reproducible evaluation.

1. Introduction

Encrypted transports restrict transparent in-network redundancy elimination. QUIC combines UDP transport, TLS security, stream multiplexing, loss recovery, and congestion control, which makes middlebox rewriting of payloads the wrong abstraction for most public or end-to-end encrypted deployments [6-10]. A WAN optimizer can only suppress redundancy that it can see, and a correctly deployed QUIC connection ensures that it sees only ciphertext. The redundancy, however, has not disappeared: registries, content-delivery networks, software-update services, and backup systems still send byte-stable objects to receivers that already hold closely related bytes from a previous transfer.

The practical question is therefore not whether repeated bytes can be replaced by references. Fixed-block reuse, rsync, and low-bandwidth file systems already demonstrate that repeated bytes can be avoided [1-5], and HTTP delta and shared-dictionary mechanisms show that receiver-held bytes can be reused as a dictionary on the web [19-22]. The question this paper addresses is whether a reference-substitution mechanism can be made explicit, individually authenticated, privacy-scoped, and compatible with encrypted endpoint streams, without claiming a universal accelerator and without asking the application to become a file-tree synchronization protocol. Figure 1 shows the resulting architecture: cooperating endpoints that already control the plaintext decide, per chunk, whether to send bytes or an authenticated reference, and the receiver reconstructs or fails closed.

We use one accounting vocabulary throughout. The effective multiplier is reconstructed or input bytes divided by encoded bytes at a stated accounting layer. Stream-payload multipliers exclude UDP/IP/link overhead; UDP-estimated multipliers add a local IPv4/UDP estimate; and congestion fairness, where claimed, applies to encoded bytes, not reconstructed bytes. This separation is deliberate: ReduLink changes how many application bytes a given number of wire bytes can reconstruct, not the physical line rate.

This paper investigates three questions. RQ1: can reference substitution over encrypted QUIC streams be made explicit, authenticated, and privacy-scoped without custom transport-layer changes? RQ2: under which workload shapes does authenticated reference substitution save bytes, and where do simpler fixed-block reuse, compression, or rsync win? RQ3: what is the component-level cost of the authentication and accounting machinery, and how stable and fair are the native QUIC results across repeated runs, scales, and emulated bottlenecks? Section 7 answers RQ2, Sections 8 and 10 answer RQ3, and Sections 4 and 5 answer RQ1.

2. Contributions and Claim Boundary

- An authenticated ReduLink reference format and validation model for endpoint-controlled encrypted streams, with per-frame binding of epoch, scope, stream id, reconstructed offset, length, nonce, and chunk identity.
- A compact binary mapping carried inside native aioquic QUIC streams, with byte-exact semantic MISS/FULL repair, repeated trials, and a payload-scaling experiment.
- An exporter-style artifact key schedule and security tests for wrong secret, scope, epoch, stream id, offset, length, replay, and tampering, including authenticated-UDP tamper and replay probes.
- A comparison against fixed-block reuse, gzip/zstd, and real rsync, including cases where these baselines win decisively, plus an explicit contrast with HTTP delta and shared-dictionary transport.
- External public source-release negative evidence, object-aligned public source-release workloads, and a public Redis-derived layer-like positive case, all hash-pinned and reproducible.
- Block-size sensitivity, component-cost measurements, competing-flow and bottleneck-emulation fairness analysis, and conservative accounting-layer separation.

The central claim is conditional. ReduLink helps when byte-stable chunks recur across warm endpoint state and authenticated reference substitution is preferable to a file-oriented delta protocol or local compression. It is not a replacement for rsync or compression, and the artifact is a native QUIC stream mapping rather than a custom QUIC extension-frame implementation. A practical example is a registry, CDN, backup, or update service that repeatedly sends object records to the same endpoint or tenant while preserving end-to-end QUIC encryption. In that setting the server can reference receiver-known objects or chunks without exposing plaintext to a middlebox; the price is authentication and metadata overhead, and the benefit is explicit context binding, safe miss repair, and a stream-compatible representation.

3. Background and Related Work

3.1 Network redundancy elimination

Spring and Wetherall introduced protocol-independent redundancy elimination, suppressing repeated byte ranges on a link by caching recently seen content at both ends [1]. Subsequent work generalized this to network-wide and coordinated redundancy elimination (SmartRE) and to enterprise and end-system services (EndRE), and measured how much redundancy real traffic contains [2-4]. These systems are powerful but assume a vantage point that can observe or terminate plaintext. ReduLink targets the opposite regime, where the transport is end-to-end encrypted and only the endpoints can act, so the redundancy decision must move into the endpoint stack.

3.2 File-delta and low-bandwidth file systems

rsync and the Low-Bandwidth File System (LBFS) show the value of transferring only the differences between file objects that both sides can name and compare [5]. They are the right tool when the workload is genuinely a file tree and both endpoints can run a delta protocol. As our negative results in Section 7.1 confirm, ordinary source-tree

updates are better served by rsync than by stream-level reference substitution. ReduLink is not aimed at this case; it is aimed at object or stream delivery where a file-tree delta negotiation is not the serving abstraction.

3.3 HTTP delta and shared-dictionary transport

The web has its own lineage of receiver-side reuse over an encrypted channel, and it is the closest existing work. RFC 3229 defines delta encoding in HTTP, sending only the difference of a response relative to a prior instance the client holds, typically using the VCDIFF format [19,20]. Shared Dictionary Compression over HTTP (SDCH) generalized this to a shared dictionary referenced across responses [21], and the recent Compression Dictionary Transport standard (RFC 9842) lets a server designate a previously fetched response as an external Brotli or Zstandard dictionary for future responses [22]. Because these mechanisms operate on HTTP responses, they already run over HTTP/3 on QUIC: receiver-held bytes are reused as a dictionary across an encrypted transport, which is precisely ReduLink's setting.

ReduLink differs in three ways that matter for the deployments it targets. First, it authenticates each reference individually and binds it to epoch, scope, stream, reconstructed offset, length, and a nonce, so a reference cannot be replayed across context or silently expanded; compression-dictionary schemes treat the dictionary as a codec input and rely on the surrounding TLS channel for integrity. Second, ReduLink defines an explicit fail-closed semantic-repair path (MISS then FULL) rather than degrading to a codec fallback. Third, it is a general stream-representation abstraction rather than a codec scoped to HTTP responses with URL-pattern dictionary matching. We therefore treat compression dictionary transport as the strongest deployed point of comparison and position ReduLink as the authenticated, fail-closed, transport-general variant of the same receiver-side reuse idea; Section 12 and Table 20 consolidate this comparison.

3.4 Content-defined chunking and deduplication privacy

Content-defined chunking determines where chunk boundaries fall and strongly affects deduplication ratio and throughput; recent surveys and fast designs analyze the trade-offs in depth [11,12]. Chunking and deduplication also create content-existence side channels: deduplication can leak whether a receiver already holds particular content, and chunk-boundary parameters can themselves be attacked [13,16,17]. These results directly motivate ReduLink's privacy scoping (Section 4) and its exclusion of global cross-user dictionaries. Modern Ethernet line rates continue to increase [14], which is why the paper avoids physical-rate claims and reports only representation-layer savings.

3.5 Deployment model

The deployment model has three roles: a sender with access to the new byte stream, a receiver with a scoped warm dictionary, and a policy domain that defines whether dictionary state is per connection, per origin, or per tenant. Public Internet mode defaults to per-connection dictionaries. Public artifact mode may use signed or manifest-controlled per-origin dictionaries. Enterprise mode may use per-tenant dictionaries with audit and quota controls. Global private cross-user dictionaries remain out of scope. Table 1 summarizes when ReduLink is and is not the right tool relative to rsync and compression.

Table 1. ReduLink deployment modes and dictionary scoping.

Deployment	Dictionary scope	Why ReduLink rather than rsync/compression
CDN or registry objects	Per origin or per client	Objects arrive as encrypted streams; file-tree delta negotiation may not be the serving abstraction.
Backup/page streams	Per endpoint or tenant	Page-like chunks recur across snapshots and can be referenced while preserving authenticated reconstruction.
Enterprise VPN/admin domain	Per tenant	Policy can allow shared warm state while keeping middleboxes away from plaintext.
Source-tree sync	File tree	Usually better served by rsync; ReduLink is not targeted here.

4. Protocol and Security Model

ReduLink is a representation layer for cooperating, mutually authenticated endpoints. QUIC packetization, packet numbers, ACK processing, TLS, and congestion control remain QUIC responsibilities; ReduLink defines only how application bytes are represented as authenticated FULL, REF, MISS, and DICT_ACK records and how the receiver validates them. The sender chunks outgoing bytes and emits FULL frames for new chunks or REF frames for chunks already believed present at the receiver. A REF is deliverable only if dictionary membership, epoch, scope, stream id, reconstructed offset, length, nonce, chunk identity, and authentication tag all validate; otherwise the receiver fails closed and requests semantic FULL repair. The full sender/receiver state machines, dictionary-admission rules, and negotiation parameters are specified in the artifact's protocol appendix.

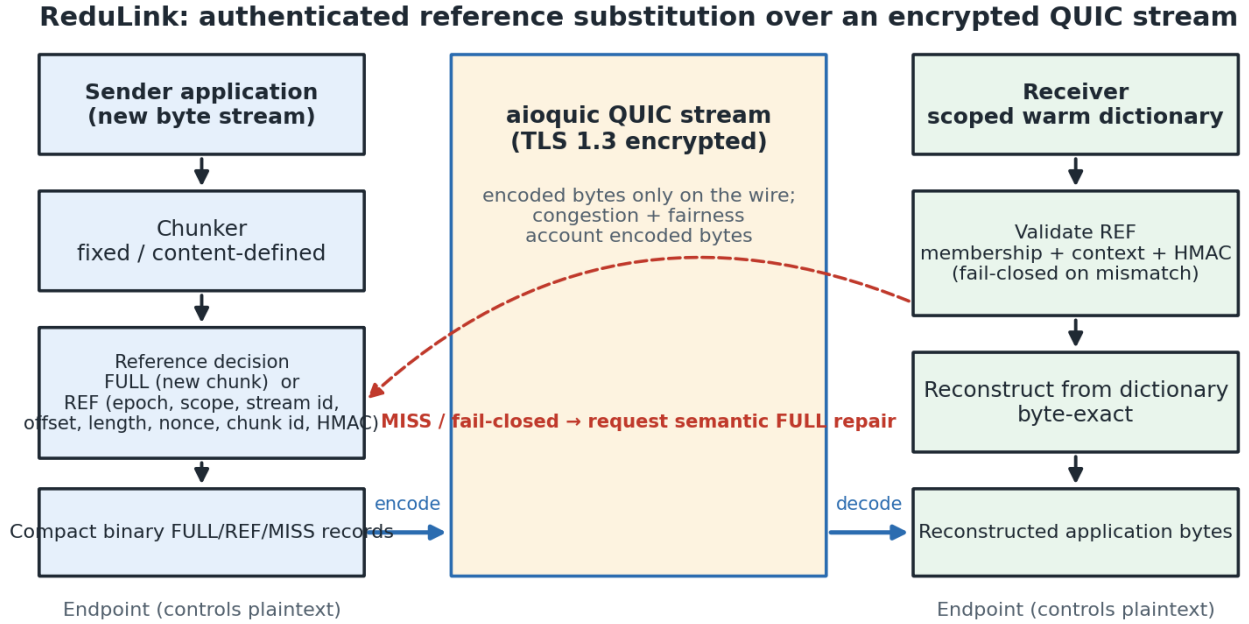


Figure 1. ReduLink protocol and deployment architecture. The sender chunks outgoing bytes and emits FULL or authenticated REF records carried as compact binary application data inside an encrypted QUIC stream; the receiver validates each reference against dictionary membership and bound context before byte-exact reconstruction, failing closed and requesting semantic FULL repair on any mismatch.

4.1 Frame types and wire model

ReduLink uses four logical frame types. A FULL frame carries chunk bytes and admits them to the receiver dictionary after validation. A REF frame carries an authenticated reference to a chunk already present at the receiver. A MISS frame is a receiver request for FULL fallback when a reference cannot be resolved. A DICT_ACK frame optionally acknowledges that a receiver admitted a chunk, letting a conservative sender reference only acknowledged chunks. Critically, the `stream_offset` field always denotes the offset in the original reconstructed application byte stream, never an offset in encoded frame bytes. Table 2 lists the bound fields. The artifact uses conservative accounting overheads of 24 bytes for FULL metadata and 32 bytes for REF metadata; these are accounting inputs, not a final QUIC wire encoding, which would recompute overhead from varint lengths and authentication-tag choice.

Table 2. ReduLink frame types and their authenticated fields.

Frame	Purpose	Bound / carried fields
FULL	Carry and admit new chunk bytes	epoch, stream id, stream offset, chunk length, chunk id, payload, auth tag
REF	Reference a chunk already at the receiver	epoch, stream id, stream offset, original length, chunk id, reference nonce, auth tag
MISS	Request FULL fallback (fail-closed)	epoch, stream id, stream offset, chunk id
DICT_ACK	Acknowledge dictionary admission	epoch, chunk id, dictionary generation

4.2 Validation and fail-closed reconstruction

The receiver applies a fixed validation order before any byte is delivered, summarized as follows:

- Reject frames whose epoch does not match the active epoch.
- For FULL, validate the authentication tag and chunk identifier, admit the payload to the dictionary, and deliver the bytes at the intended stream offset.
- For REF, validate the authentication tag, expansion bound, stream offset, original length, nonce, and dictionary presence; deliver reconstructed bytes only after all checks succeed.
- Emit MISS when the referenced chunk is unavailable or policy refuses the reference; a REF that cannot become valid in the current epoch triggers an immediate MISS.
- Apply stream ordering and flow control to reconstructed bytes, while applying congestion accounting to transmitted wire bytes.

A reference miss is treated as a synchronization failure, not a corruption event: the receiver delivers no reconstructed bytes, emits MISS, and the sender repairs with a FULL for the same stream id, offset, length, and chunk id. A REF whose original_length disagrees with the stored chunk length, or that would expand beyond the negotiated bound, is a hard validation failure. Section 11 reports prototypes that exercise exactly these miss, tamper, and replay paths.

4.3 Dictionaries, epochs, and key schedule

Dictionaries are scoped per connection by default and per origin or per tenant only under explicit policy; global cross-user dictionaries are out of scope. Implementations use bounded LRU-style eviction, per-origin or per-tenant quotas, and short-lived epochs, and a receiver may refuse admission under memory pressure or policy. Chunk identifiers are computed with domain separation over the active epoch, dictionary scope, stream context where applicable, a canonical length encoding, and the chunk bytes, so the same bytes in a different context yield a different identifier. The artifact derives inner authentication keys with an HKDF exporter-style schedule keyed by an explicit experiment secret plus ALPN, epoch, scope, connection, and stream context; tests confirm that the derived secret changes when any of these inputs changes. A production profile should instead key this schedule from QUIC TLS exporter bytes.

4.4 Threat model

Table 3 maps each security property to the mechanism it requires and to the current artifact status, and Table 4 names the privacy modes and their leakage risks. The artifact implements HMAC binding, chunk-id validation, nonce rejection, length and expansion checks, and fail-closed repair; it does not implement production exporter-derived keys, custom extension-frame parsing, production replay windows, or cross-tenant isolation enforcement, which are stated as future work. Like other deduplication systems, ReduLink can create a content-existence oracle when an adversary can choose payloads or references and observe transfer size, timing, or MISS behavior; per-connection dictionaries remove the cross-user oracle in public mode, and shared dictionaries are permitted only for public artifacts or explicit trust domains.

Table 3. Security properties, required mechanisms, and artifact status.

Property	Required mechanism	Artifact status
Integrity	FULL/REF authentication, chunk-id, offset and length binding	Modeled by reconstruction and mismatch tests; production crypto pending.
Context binding	Exporter-derived keys, epoch/stream/offset/scope, nonce, replay window	HMAC binding and nonce rejection implemented; exporter keys pending.
Dictionary safety	Authenticated FULL, manifest commitment, admission/eviction policy	FULL chunk-id checks modeled; manifest policy not implemented.
Expansion bound	Per-frame, per-stream, per-epoch reconstructed-byte caps	Length and accounting checks modeled; full flow control pending.
Privacy scope	Per-connection default, no global cross-user dictionary	Policy specified; cross-tenant enforcement not implemented.

Table 4. Privacy modes and leakage risks.

Mode	Dictionary scope	Leakage risk	Default
------	------------------	--------------	---------

Public Internet	Per connection only	Same-connection access-pattern leakage	Yes
Public artifacts	Per-origin signed manifest	Artifact-version / popularity inference	Optional
Enterprise VPN	Tenant / admin domain	Intra-tenant content-existence leakage	Optional
CDN/update channel	Origin-scoped public versions	Version-possession inference	Optional
Global cross-user	Any user	Private content-possession leakage	No (out of scope)

5. Implementation and Metrics

The artifact is implemented in Python for reproducibility and uses aioquic for native QUIC stream experiments [18]. ReduLink messages are compact binary application-stream records inside QUIC streams; this exercises a real QUIC handshake and encryption but does not implement custom QUIC frame types or transport parameters. The JSON encoding is retained only as a readable baseline, and the default path uses the compact binary format, which encodes frame type, stream id, reconstructed offset, length, chunk id, nonce, authentication tag, and payload only where required.

Three byte layers are reported. Input bytes are the reconstructed application bytes. Stream-payload bytes are the ReduLink or raw application bytes written into QUIC streams. UDP-estimated bytes add a local IPv4/UDP estimate from the loopback proxy path. The paper avoids treating stream-payload multipliers as full wire-rate measurements; they isolate the representation layer so that raw QUIC, ReduLink, gzip, zstd, fixed reuse, and rsync-style baselines can be compared consistently. Table 5 records, for each evidence layer, what it supports and what it does not prove.

Two reproducibility notes apply to the absolute numbers. Byte multipliers are deterministic functions of the input bytes and chunking parameters and are therefore machine-independent; they are the paper's results. Wall-clock times and local throughputs (for example the client elapsed times in Section 8 and the MiB/s figures in Section 9) are indicative measurements from a single commodity multi-core Linux host and are hardware-dependent; they should be read as order-of-magnitude component costs, not as portable performance guarantees.

Table 5. Evidence hierarchy: what each layer supports and does not prove.

Evidence	Supports	Does not prove
Model	FULL/REF accounting and reconstruction	Transport behavior
Secure model	HMAC binding and replay checks	Live QUIC exporter use
aioquic stream	Encrypted QUIC stream mapping	Custom extension frames
UDP/IPv4 estimate	Local datagram accounting	Full packet capture
Workloads	Workload sensitivity	Universal acceleration

6. Evaluation Methodology

Evaluation uses four classes of evidence. First, deterministic journal fixtures (disk snapshot, OCI layer, package metadata, repository snapshot, structured logs, and a compressed negative control) isolate predicted positive and negative workload shapes. Second, public source-release pairs (Click, Redis, nginx) test ordinary source-tree updates where ReduLink should not be assumed to help. Third, object-aligned public release experiments use the same public bytes but model registry/CDN object delivery rather than tarball synchronization. Fourth, native aioquic stream experiments measure the encrypted stream mapping on positive and negative cases, under loss, at scale, and against competing flows and emulated bottlenecks.

For each workload the paper reports ReduLink fixed chunking, ReduLink content-defined chunking where relevant, fixed-block reuse, compression, and rsync where the baseline is structurally applicable. The fixed-block baseline is intentionally strong and simple: if it beats ReduLink, the result is reported rather than hidden, because ReduLink is a security and transport-compatibility layer over reuse, not a superior matching algorithm. All external corpora are hash-pinned, and every figure and table is regenerated from committed result files.

7. Workload Results and Baseline Interpretation

Table 6 reports the deterministic warm-state fixtures. On most byte-stable workloads, fixed-block reuse matches or beats ReduLink; the distinct ReduLink contribution is not superior matching but authenticated, scoped, stream-compatible reference substitution with explicit repair and privacy rules. The negative rows (package metadata, logs, and the compressed control) are included precisely because they fail: where repetition is local to one object, compression wins, and where there is no reusable structure, both fixed reuse and ReduLink correctly decline to help.

Table 6. Warm-state workload results (effective stream-payload multiplier): ReduLink versus fixed-block reuse and best compression.

Workload	ReduLink	Fixed reuse	Best compression	RL - fixed
disk snap	28.49x	31.97x	1.06x	-3.48x
oci layer	11.05x	10.68x	1.50x	+0.37x
package meta	0.99x	9.88x	14.31x	-8.89x
repository snap	24.73x	24.60x	10.37x	+0.14x
logs	0.99x	14.89x	18.01x	-13.89x
compressed neg	0.99x	1.00x	1.00x	-0.01x

7.1 External Public Source Releases

Table 7 reports three independent, hash-pinned source-release pairs. The result is negative for ReduLink and strongly positive for rsync. This is an important and deliberately reported finding: ordinary related source trees are better served by file-oriented delta transfer than by stream-level reference substitution, because rsync can exploit fine-grained intra-file similarity that whole-object referencing cannot.

Table 7. External public source-release pairs: ReduLink and fixed reuse versus real rsync (negative case for ReduLink).

Public pair	ReduLink	Fixed reuse	rsync total
click-8.1.7 to 8.1.8	0.99x	0.99x	6.48x
redis-7.2.4 to 7.2.5	1.00x	1.00x	73.13x
nginx-1.25.3 to 1.25.4	1.00x	0.99x	49.49x

7.2 Object-Aligned Public Release Transfers

Table 8 and Figure 2 use the same public release tarballs but change the transfer abstraction: files are treated as object records and chunked independently, modeling registry/CDN object delivery rather than source-tree synchronization. Redis and nginx show positive ReduLink results because many public release objects remain byte-identical across versions; Click is weaker because fewer files are unchanged. ReduLink tracks the idealized fixed-object-reuse baseline within a few percent, and the authenticated variant costs only a few additional percent. This is a stronger external positive signal than a purely synthetic corpus, though it is still a transfer-model experiment rather than a captured production trace.

Table 8. Object-aligned public release transfers modeling registry/CDN object delivery.

Public release	File stability	Bytes	ReduLink	Secure RL	Fixed object reuse	gzip
click	80/144 unchanged	947,826	1.94x	1.91x	1.94x	2.86x
redis	1557/1582 unchanged	15,467,095	9.70x	9.04x	9.68x	4.61x
nginx	468/503 unchanged	7,357,392	4.38x	4.25x	4.38x	6.09x

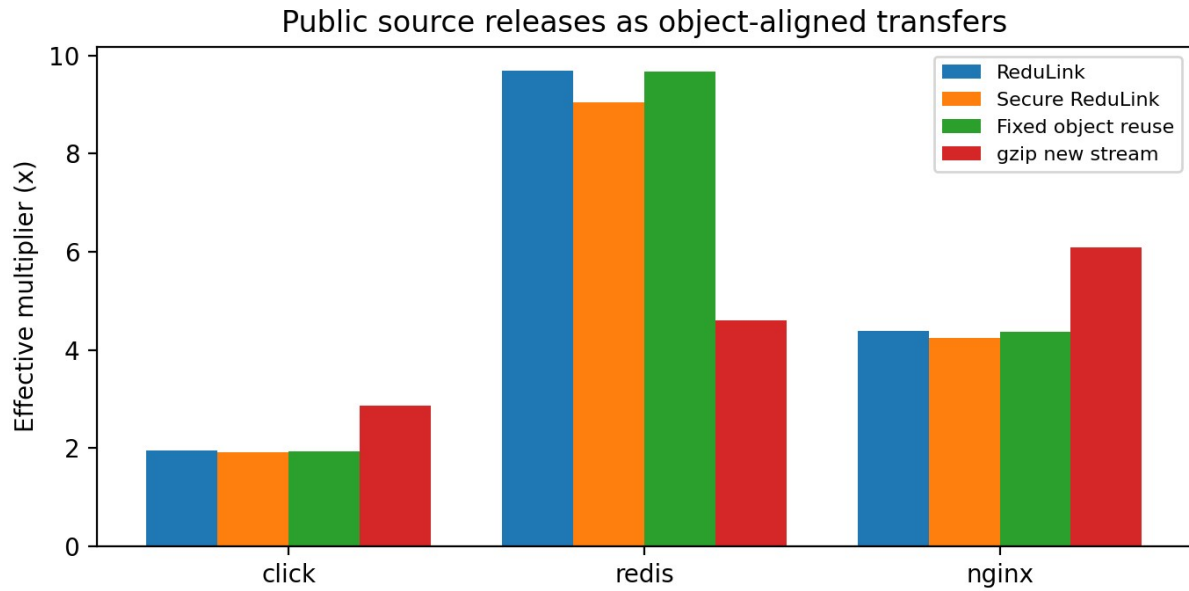


Figure 2. Object-aligned public release transfers: effective stream-payload multipliers for ReduLink, the authenticated variant, fixed object reuse, and gzip.

7.3 Layer-Like External-Positive Case

Table 9 reports a Redis-derived layer-like case built from included public Redis release bytes and aligned changed blocks. Its role is to test the predicted positive case for layer-like objects where stable chunks remain aligned; it is not a production trace. Here ReduLink reaches 21.37x with the authenticated variant at 18.96x, again close to the fixed-reuse reference.

Table 9. Layer-like external-positive case derived from public Redis release bytes.

Case	Input bytes	ReduLink	Secure RL	Fixed reuse
redis-layered-public-positive	524,288	21.37x	18.96x	21.33x

7.4 Block-Size Sensitivity

Table 10 and Figure 3 show that the best block size is workload-dependent: smaller blocks capture more aligned reuse but pay more per-chunk overhead, while larger blocks lose alignment. The artifact therefore treats 4 KiB as a reproducible default rather than a universal optimum, and a production deployment would tune the chunk size per workload class.

Table 10. Block-size sensitivity of the effective multiplier (fixed chunking).

Workload	1 KiB	2 KiB	4 KiB	8 KiB	16 KiB
disk snapshot	17.13x	23.33x	28.49x	17.08x	8.99x
oci layer	12.11x	11.86x	11.05x	5.91x	3.60x
repository snapshot	20.21x	24.99x	24.73x	14.45x	7.55x

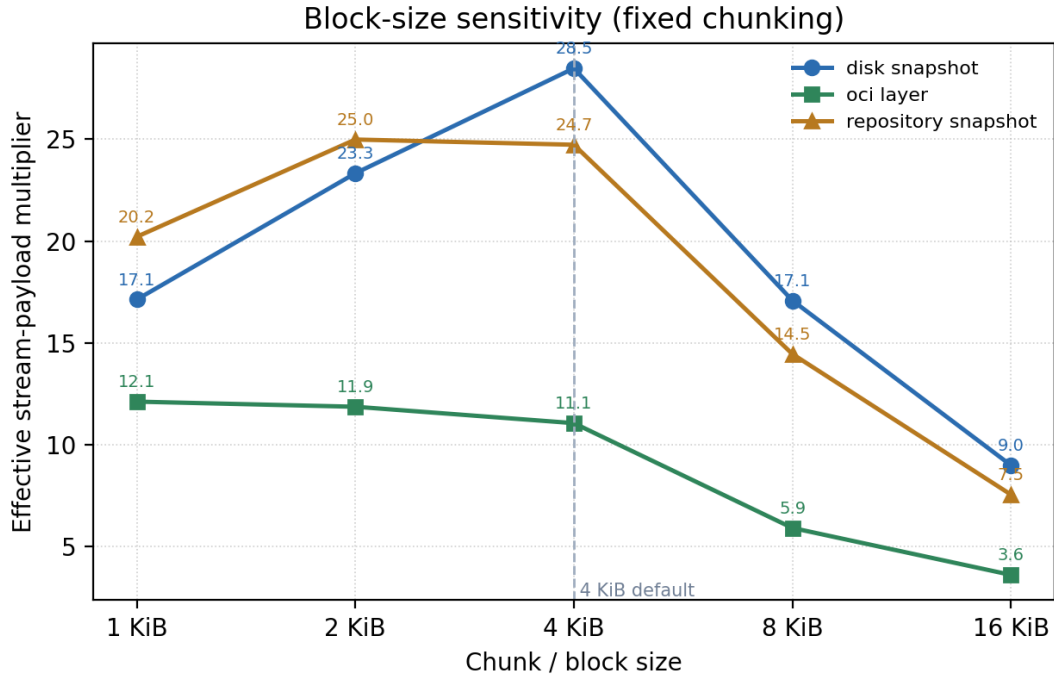


Figure 3. Block-size sensitivity of the effective multiplier for fixed chunking across three warm-update fixtures.

8. Native QUIC Experiments

8.1 Stream mapping under loss

Table 11 compares raw and ReduLink stream mapping at zero and periodic datagram loss, and Table 12 reports positive and negative workload cases. ReduLink reconstructs byte-exactly in every case, including the lossy proxy path, and the compressed-negative control correctly yields no gain. Figure 4 plots the workload cases. Because ReduLink writes fewer stream-payload bytes, its UDP-estimated multiplier stays above one on positive workloads even after the IPv4/UDP per-datagram overhead is added.

Table 11. Native aioquic stream mapping: raw versus ReduLink under zero and periodic datagram loss.

Method	Loss every	Stream x	UDP-est x	Recon.
Raw	0	1.00x	0.90x	OK
ReduLink	0	3.19x	2.60x	OK
Raw	9	1.00x	0.78x	OK
ReduLink	9	3.19x	2.38x	OK

Table 12. Native aioquic stream mapping across positive and negative workload cases.

Case	Input	Stream x	UDP-est x	Misses	Recon.
demo positive	98,304	3.19x	2.57x	13	OK
compressed negative	262,242	0.91x	0.84x	0	OK
Redis layered	524,288	3.83x	3.46x	72	OK

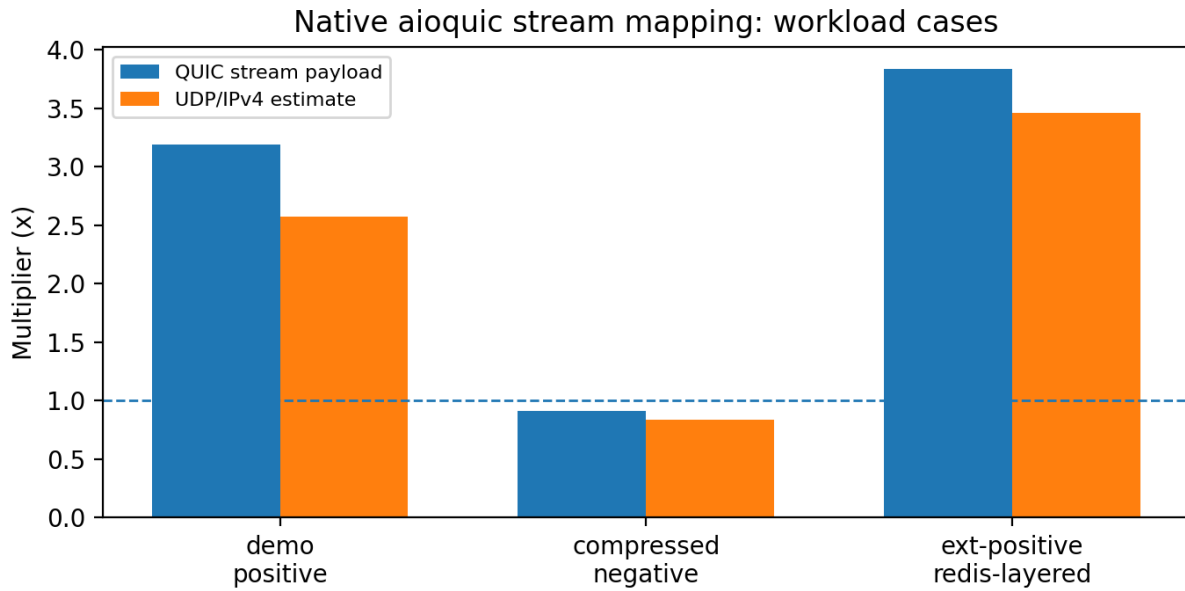


Figure 4. Native aioquic stream mapping on positive and negative workload cases.

8.2 Repeated trials and scaling

Table 13 reports three repeated trials of the native demo workload. The multiplier is deterministic for fixed bytes, while elapsed times vary modestly across runs; reporting both avoids cherry-picking a single favorable QUIC run. Table 14 then scales the payload from 96 to 1024 blocks: the stream multiplier rises from 3.19x to 3.87x as more reusable chunks accumulate, the miss count grows proportionally with the dictionary warm-up, and reconstruction remains byte-exact at every scale.

Table 13. Repeated native QUIC trials (n = 3): run-to-run stability of multipliers and elapsed time.

Metric	n	Mean	Std. dev.	Min	Max
raw client ms	3	227.043667	19.900327	205.943000	245.474000
redulink stream multiplier	3	3.189928	0.000000	3.189928	3.189928
redulink udp est multiplier	3	2.578853	0.003857	2.574954	2.582666
redulink client ms	3	137.076000	4.472720	131.991000	140.401000

Table 14. Payload scaling of the native aioquic stream mapping.

Blocks	Input bytes	Stream x	Misses	Client ms	Recon.
96	98,304	3.19x	13	124.0	OK
512	524,288	3.77x	73	192.9	OK
1024	1,048,576	3.87x	146	260.4	OK

9. Component Costs

Table 15 reports component-level throughput on the reference host. Fixed chunking, HMAC validation, and compact binary encoding are fast; the Python content-defined chunker is slow and is explicitly not production line-rate evidence. A production implementation would require optimized native chunking and tighter integration with the transport stack. As noted in Section 5, these throughputs are hardware-dependent component costs, not portable guarantees.

Table 15. Component-level cost measurements (local Python artifact timing; not production line-rate).

Component	Input bytes	MiB/s local	Roundtrip
fixed chunking	2,097,152	6237.0	not applicable
CDC chunking	2,097,152	7.5	not applicable

model fixed roundtrip	1,573,964	680.0	OK
HMAC roundtrip	1,573,964	192.2	OK
wire encode	1,573,964	4696.9	not applicable
wire decode	1,573,964	2451.2	not applicable

10. Fairness and Accounting

The central fairness rule is that ReduLink must not receive congestion credit for reconstructed bytes: congestion control and bottleneck service account encoded wire bytes only. Table 16 reports the wire-byte accounting experiment, in which ReduLink's encoded wire share is 0.083 against the raw stream's 0.917 over 16 rounds, while still reconstructing the same application bytes; the effective application multiplier is 10.72x. This confirms that the savings appear as fewer encoded bytes, which is the only place a fair transport should credit them.

Table 16. Wire-byte fairness accounting: ReduLink is charged on encoded bytes, not reconstructed bytes.

Metric	Value
Fairness rule	Congestion and bottleneck service count encoded wire bytes
Raw encoded wire share	0.917
ReduLink encoded wire share	0.083
ReduLink uses less wire than raw	yes
Effective application multiplier	10.72x
Rounds	16

Table 17 reports a concurrent competing-flow smoke test in which a raw QUIC flow and a ReduLink flow share a localhost schedule. Jain's fairness index on encoded rates is 0.919 and on reconstructed rates is 0.914, with all flows reconstructing correctly; this is a smoke test, not a full congestion-control study. Table 18 then emulates a fluid fair-share bottleneck across rate and RTT points using the measured encoded stream bytes. Because ReduLink transmits fewer encoded bytes, it completes sooner and delivers higher application goodput at the same encoded fair share (both flows deliver the same 98,304 application bytes, so the goodput columns are directly comparable); for example at 25 Mbps and 10 ms it completes in 29.7 ms versus 51.3 ms for raw, delivering 26.5 Mbps of reconstructed goodput. The benefit narrows as the path becomes RTT-bound, as the 50 ms rows show, because latency rather than encoded volume then dominates completion time.

Table 17. Concurrent competing-flow smoke test (Jain fairness index).

Metric	Value
Scenario	2 concurrent rounds, ~25.0 Mbps hint, loss every 9
Encoded-rate Jain index	0.919
Reconstructed-rate Jain index	0.914
All flows reconstructed	yes
Scope	localhost concurrent smoke test, not a full congestion-control study

Table 18. Fluid fair-share bottleneck emulation over measured encoded stream bytes (raw versus ReduLink).

Rate Mbps	RTT ms	Raw compl. ms	RL compl. ms	Raw app goodput Mbps	RL app goodput Mbps
10.0	10.0	113.3	59.3	6.9	13.3
10.0	50.0	153.3	99.3	5.1	7.9
25.0	10.0	51.3	29.7	15.3	26.5
25.0	50.0	91.3	69.7	8.6	11.3
100.0	10.0	20.3	14.9	38.7	52.7
100.0	50.0	60.3	54.9	13.0	14.3

The package does not include a full tc/netem or Mininet congestion-control study; the competing-flow and bottleneck results are deliberately conservative emulations that isolate the encoded-byte accounting principle rather than claim production fairness.

11. Repair and Authentication Prototypes

Beyond the in-stream experiments, three prototypes exercise the fail-closed and authentication paths directly. Table 19 summarizes them. The semantic-repair demo forces a 25 percent dictionary-miss fraction and confirms that every miss is repaired with a FULL and that reconstruction is byte-exact. The localhost UDP repair experiment drops every seventh data datagram and uses timeout-based retransmission, again reconstructing fully after 15 semantic repairs and 6 retransmissions. The authenticated UDP experiment adds explicit negative probes: a tampered authentication tag and a replayed nonce are both rejected (1 tamper and 1 replay rejection) before normal authenticated repair traffic is accepted, demonstrating fail-closed behavior under active manipulation.

Table 19. Repair and authentication prototypes (localhost).

Prototype	Input bytes	Misses / repairs	Negative probes rejected	Recon.
Semantic repair demo	32,546	2 / 2	n/a (25% missing)	OK
UDP repair (drop every 7)	49,152	15 / 15	6 retransmits	OK
Authenticated UDP	65,536	12 / 12	1 tamper, 1 replay	OK

12. Why Not Just Compress or Use rsync?

Compression is best when repetition is local to one object; rsync is best when both endpoints share a file tree or file object and can run a delta protocol; and HTTP delta or shared-dictionary transport is best when the application is HTTP and a codec-level dictionary suffices. The ReduLink use case is different: encrypted endpoint streams where dictionary state is already available at the receiver, the transfer is not naturally a file-tree synchronization session, and each reconstruction step must be individually authenticated and fail closed. The evaluation confirms the boundary. On source-release trees, rsync dominates (Table 7). On package metadata and logs, compression or fixed-block reuse dominates (Table 6). On object-aligned public releases and layer-like byte-stable objects (Tables 8 and 9), ReduLink provides authenticated reference substitution at modest overhead over a simple fixed-reference baseline.

The motivating cases are therefore not arbitrary file synchronization. They are stream-serving cases in which the endpoint already knows that a receiver has dictionary state but must still authenticate every reconstruction step and preserve encrypted transport semantics. A CDN edge, registry, backup client, or enterprise service may choose this representation because it avoids exposing plaintext to a WAN optimizer and avoids changing the application protocol into a file-tree delta session, while keeping a stronger per-reference integrity guarantee than a codec-level shared dictionary provides.

Table 20 consolidates the positioning. ReduLink is the only approach in the comparison that combines operation over an end-to-end encrypted QUIC stream with per-reference authentication and an explicit fail-closed repair path; the other approaches each win in their own regime, which is exactly why ReduLink is presented as a complementary representation layer rather than a replacement.

Table 20. Positioning of ReduLink relative to related redundancy-suppression approaches.

Approach	E2E-encrypted transport	Per-reference auth	Fail-closed repair	Receiver dictionary	Best-fit workload
In-network RE [1-4]	No (needs plaintext vantage)	No	n/a	Network cache	High-redundancy unencrypted links
rsync / LBFS [5]	Endpoints only	Transport integrity only	n/a (delta negotiation)	File/object index	File-tree synchronization
HTTP shared dictionary / CDT [19-22]	Yes (HTTP/3 on QUIC)	No (TLS channel only)	No (codec fallback)	Prior HTTP response	HTTP responses with prior versions
Local compression (gzip/zstd)	Yes	No	n/a	None (intra-object)	Locally repetitive single objects
ReduLink (this work)	Yes (QUIC streams)	Yes (per reference)	Yes (MISS then FULL)	Scoped warm dictionary	Warm-state object / layer transfers

13. Limitations and Future Work

The implementation maps ReduLink into QUIC streams rather than adding custom QUIC extension frames or transport parameters, and it uses an exporter-style key schedule model rather than live QUIC TLS exporter bytes. The public object-sequence and layer-like workloads are derived from real release bytes but are transfer-model experiments rather than captured production traces. The fairness evidence (competing-flow Jain indices and fluid bottleneck emulation in Section 10) is conservative and does not replace a full network-emulator congestion-control study, and all transport experiments run over localhost with a deterministic loss proxy rather than over a real path with diverse RTT, bandwidth, and loss. Absolute timings are hardware-dependent.

Future work should add custom QUIC frame integration and transport-parameter negotiation, live exporter-derived keys, production replay-window and cross-tenant isolation enforcement, tc/netem or Mininet fairness experiments with competing real flows, packet capture, larger and independently captured registry or backup traces, and a direct head-to-head comparison against Compression Dictionary Transport on the same corpora.

14. Reproducibility

The package contains source code, public corpora, generated corpora, benchmark scripts, result CSV/JSON files, figures, a citation checker, and tests [15]. The reviewer-facing smoke command is `python3 scripts/run_smoke_validation.py` and full validation is `python3 scripts/run_full_validation.py`. aioquic tests skip gracefully when aioquic is unavailable; installing `requirements-dev.txt` enables complete QUIC stream validation. Every table and figure in this paper is regenerated from the committed result files by `scripts/build_manuscript_v3_0.py` and `scripts/make_journal_figures_v2_8.py`.

15. Conclusion

ReduLink is best understood as authenticated, scoped reference substitution for encrypted endpoint streams. Its byte savings are conditional and are often reproduced or exceeded by simpler fixed-block reuse, compression, or rsync; the contribution is to make reference substitution explicit, individually authenticated, privacy-scoped, repairable, and compatible with native QUIC stream transport, and to position it precisely against both in-network redundancy elimination and HTTP shared-dictionary transport. The evidence supports ReduLink for warm-state object-aligned or layer-like transfers, while negative source-release results show where it should not be used.

Data and Code Availability

All source code, public-corpus manifests, generated corpora, benchmark scripts, result CSV/JSON files, figures, the citation checker, and the test suite are openly available in the ReduLink repository [15] under the MIT License. Every figure and table is regenerated from the committed result files, so the reported numbers can be reproduced from the artifact. Large external public-release corpora are reconstructed from documented public sources rather than redistributed.

References

- [1] N. T. Spring and D. Wetherall, 'A protocol-independent technique for eliminating redundant network traffic,' ACM SIGCOMM, 2000.
- [2] A. Anand, V. Sekar, and A. Akella, 'SmartRE: An architecture for coordinated network-wide redundancy elimination,' ACM SIGCOMM, 2009.
- [3] A. Anand et al., 'Redundancy in network traffic: Findings and implications,' ACM SIGMETRICS, 2009.
- [4] B. Aggarwal et al., 'EndRE: An end-system redundancy elimination service for enterprises,' USENIX NSDI, 2010.
- [5] A. Muthitacharoen, B. Chen, and D. Mazieres, 'A low-bandwidth network file system,' ACM SOSP, 2001.
- [6] J. Iyengar and M. Thomson, 'QUIC: A UDP-based multiplexed and secure transport,' RFC 9000, IETF, 2021.
- [7] M. Thomson and S. Turner, 'Using TLS to secure QUIC,' RFC 9001, IETF, 2021.
- [8] J. Iyengar and I. Swett, 'QUIC loss detection and congestion control,' RFC 9002, IETF, 2021.
- [9] M. Kuehlewind and B. Trammell, 'Applicability of the QUIC transport protocol,' RFC 9308, IETF, 2022.
- [10] B. Trammell et al., 'Manageability of the QUIC transport protocol,' RFC 9312, IETF, 2022.
- [11] W. Xia, X. Zou, Y. Zhou, H. Jiang, C. Liu, D. Feng, Y. Hua, Y. Hu, and Y. Zhang, 'The design of fast content-defined chunking for data deduplication based storage systems,' IEEE Transactions on Parallel and Distributed Systems, vol. 31, no. 9, 2020.

- [12] M. Gregoriadis, L. Balduf, B. Scheuermann, and J. Pouwelse, 'A thorough investigation of content-defined chunking algorithms for data deduplication,' arXiv:2409.06066, 2024.
- [13] B. Alexeev, C. Percival, and Y. X. Zhang, 'Chunking attacks on file backup services using content-defined chunking,' arXiv:2504.02095, 2025.
- [14] IEEE 802.3 Ethernet Working Group, 'IEEE 802.3 Ethernet Working Group active projects,' accessed June 2026.
- [15] M. Nguyen, 'ReduLink artifact and reproducibility package,' GitHub repository, 2026. <https://github.com/pinkysworld/redulink-deduplex-quick>
- [16] D. Harnik, B. Pinkas, and A. Shulman-Peleg, 'Side channels in cloud services: Deduplication in cloud storage,' IEEE Security and Privacy, 2010.
- [17] M. Bellare, S. Keelveedhi, and T. Ristenpart, 'DupLESS: Server-aided encryption for deduplicated storage,' USENIX Security, 2013.
- [18] aioquic project contributors, 'aioquic: QUIC and HTTP/3 implementation in Python,' software repository, version 1.3.0, 2026. <https://github.com/aiohttp/aioquic>
- [19] J. Mogul, B. Krishnamurthy, F. Douglass, A. Feldmann, Y. Goland, A. van Hoff, and D. Hellerstein, 'Delta encoding in HTTP,' RFC 3229, IETF, 2002.
- [20] D. Korn, J. MacDonald, J. Mogul, and K. Vo, 'The VCDIFF generic differencing and compression data format,' RFC 3284, IETF, 2002.
- [21] J. Butler, W.-H. Lee, B. McQuade, and K. Mixer, 'A proposal for shared dictionary compression over HTTP (SDCH),' IETF Internet-Draft, 2008.
- [22] P. Meenan and Y. Weiss, 'Compression dictionary transport,' RFC 9842, IETF, 2025.